

Universidad Internacional del Ecuador



Ingeniería en software

Nombre:

Daniela Paillacho

Asignatura:

PROGRAMACION ORIENTADA A OBJETOS

Fecha de envió: 09/02/26

Fecha de entrega:15/02/26

Quito-Ecuador

2025-2026

Paso 1: Continuación del plan

I. Codificación inicial del sistema

Implementar las funcionalidades básicas: registro de libros, usuarios y préstamos.

II. Identificación de clases principales

- **Libro** (atributos: título, autor, ISBN, formato electrónico, disponibilidad).
- **Usuario** (atributos: nombre, correo, historial de préstamos).
- **Prestamo** (atributos: fecha inicio, fecha fin, estado).
- **Biblioteca** o **GestorLibros** (métodos para agregar, buscar, eliminar libros).

Paso 2: Desarrollo del sistema

III. Implementación de encapsulación

- Proteger atributos con **private**.
- Usar **getters** y **setters**.

IV. Funcionalidades más complejas para su adecuado entendimiento.

```
class Libro:
    def __init__(self, titulo, autor, categoria, archivo):
        self.__titulo = titulo
        self.__autor = autor
        self.__categoria = categoria
        self.__archivo = archivo
        self.__disponible = True

    # Getters y setters

    def get_titulo(self):
        return self.__titulo

    def set_titulo(self, nuevo_titulo):
        self.__titulo = nuevo_titulo

    def is_disponible(self):
        return self.__disponible

    def prestar(self):
        if self.__disponible:
            self.__disponible = False
        else:
            raise Exception("El libro no está disponible")
```

- Préstamo de libro con validación

```
1  def prestar_libro(libro, usuario):
2      """
3      Este método gestiona el préstamo de un libro a un usuario.
4      Verifica si el libro está disponible antes de asignarlo.
5      Si no está disponible, lanza una excepción para evitar duplicidad.
6      """
7      if libro.is_disponible():
8          libro.prestar()
9          nuevo_prestamo = Prestamo(usuario, libro)
10         usuario.historialPrestamos.append(nuevo_prestamo)
11     else:
12         raise Exception("El libro no está disponible para préstamo.")
```

➤ Búsqueda avanzada por título

```
1  def buscar_libro_por_titulo(titulo, lista_libros):
2      """
3      Realiza una búsqueda exacta e insensible a mayúsculas/minúsculas.
4      Devuelve una lista de libros que coincidan con el título ingresado.
5      """
6      resultados = []
7      for libro in lista_libros:
8          if libro.get_titulo().lower() == titulo.lower():
9              resultados.append(libro)
10     return resultados
```

➤ Autenticación de usuario

```
1  def autenticar_usuario(correo, contraseña, lista_usuarios):
2      """
3      Verifica si el correo y la contraseña coinciden con algún usuario registrado.
4      Utiliza Werkzeug para comparar contraseñas cifradas.
5      Si no hay coincidencia, devuelve None.
6      """
7      for usuario in lista_usuarios:
8          if usuario.correo == correo and check_password_hash(usuario.contraseña_hash, contraseña):
9              return usuario
10     return None
```

Lectura de archivo PDF

```
1  def extraer_texto_pdf(ruta_archivo):
2      """
3      Utiliza PyPDF2 para abrir y leer el contenido de un archivo PDF.
4      Extrae el texto de todas las páginas y lo concatena en una sola cadena.
5      Maneja errores si el archivo está corrupto o no se puede abrir.
6      """
7      try:
8          with open(ruta_archivo, 'rb') as archivo:
9              lector = PyPDF2.PdfReader(archivo)
10             texto = ""
11             for pagina in lector.pages:
12                 texto += pagina.extract_text()
13             return texto
14     except Exception as e:
15         print(f"Error al leer el archivo PDF: {e}")
16         return ""
```